

Handling Terraform State File Corruption

Diagnosis, recovery paths, and prevention

Don't run `apply` or `destroy` while state is in a bad state. A corrupted or inconsistent state file is exactly the situation where a blind `apply` can do real damage — Terraform might think resources don't exist and try to recreate them, colliding with what's already there. Stop and diagnose first.

Step 1 • Check for a stuck lock

If the error is about acquiring a lock rather than the file itself being unreadable:

```
terraform plan
# Error: Error acquiring the state lock ... Lock Info: ID: xxxxx
```

Confirm no other process (a hung CI job, someone's laptop) is actually mid-run, then release it:

```
terraform force-unlock <LOCK_ID>
```

Only do this once you're sure nothing else is genuinely holding the lock — force-unlocking while another `apply` is truly in progress is how you get real corruption.

Step 2 • Assess whether the state file itself is actually broken

```
terraform state list
terraform show
```

If these fail with a JSON parse error, the file is genuinely malformed. If they work but the content looks wrong (missing resources, stale data), it's a sync issue rather than corruption — see Step 5.

Step 3 • Restore from a backup

Local state

Terraform automatically writes `terraform.tfstate.backup` before every state-modifying operation — this is your first port of call:

```
cp terraform.tfstate.backup terraform.tfstate
terraform plan # should show no unexpected diff if this is the right version
```

S3 backend

If bucket versioning was enabled (it should always be, on a state bucket), pull the previous good version:

```
aws s3api list-object-versions --bucket my-tf-state-bucket --prefix path/to/state/terraform.tfstate

aws s3api get-object --bucket my-tf-state-bucket \
  --key path/to/state/terraform.tfstate \
  --version-id <PREVIOUS_VERSION_ID> \
  restored-state.json

terraform state push restored-state.json
```

Terraform Cloud / Enterprise

State history is versioned automatically in the UI — go to the workspace's States tab and roll back to a prior version directly, no CLI needed.

Always back up the current (possibly bad) state before overwriting anything: `terraform state pull > state-before-fix-$(date +%F).json`

Step 4 • If no good backup exists, rebuild state via import

This is the tedious-but-reliable fallback. For each resource that already exists in real infrastructure but is missing/wrong in state:

```
terraform import aws_instance.web i-0123456789abcdef0
terraform import aws_eks_cluster.this test-cluster
```

You need the resource addresses from your `.tf` files and the real-world IDs from the console/CLI. After importing everything, run `terraform plan` — it should show no changes (or only genuinely intended ones) once state matches reality.

Step 5 · For state that's just drifted, not corrupted

If the file parses fine but doesn't reflect real infrastructure (someone changed something outside Terraform), you don't need to restore anything — just refresh:

```
terraform apply -refresh-only
```

This shows you a plan of what Terraform would update in state to match reality, without touching actual infrastructure, and you approve it explicitly rather than it happening silently the way the old `terraform refresh` did.

Small surgical fixes without a full restore

If only one or two resources are the problem (a bad entry, a resource that needs to move addresses after a refactor), you often don't need to touch the raw JSON at all:

```
terraform state rm aws_instance.old_broken_entry      # remove a stale/bad entry (doesn't destroy real infra)
terraform state mv aws_instance.old_name aws_instance.new_name # rename after refactoring .tf code
```

Prevention, for next time

- **Use a remote backend with locking** — S3 + a DynamoDB lock table, or Terraform Cloud — rather than local state. This is what prevents concurrent-write corruption in the first place.
- **Enable versioning on the state bucket** so you always have prior versions to roll back to.
- **Never hand-edit the state JSON directly** — always go through `terraform state <subcommand>` or `pull/push` so Terraform validates the format.
- **Gate applies through CI with locking** rather than letting multiple people run `terraform apply` from their laptops against the same state.